

Your blockchain prototype, explained

SIH Member 5 | Repository edition v0.2.0 | 13 September 2026

The one-sentence explanation

We keep detailed screening events in a private audit journal and store their hash commitments in a smart contract, so later changes to anchored history can be detected against the trusted ledger.

This guide follows the code already built for your SIH document-screening project and its 27-cell Colab notebook. It explains the purpose first, then the technologies, the code path, team integration and future improvements. Examples use synthetic data.

What problem are we solving?

Suppose the system flags a document with risk score 78. Later, someone changes the stored decision to accept. Investigators need to know what was originally recorded, who the backend said acted, and whether the record has changed. Your module protects that recorded history; Members 1-4 assess the document itself.

Built in this prototype	Still needed for deployment
Separate Python audit module, Solidity contract and EVM connector.	Connection to the rewritten backend and its durable integration jobs.
Signed transactions and real receipts on a local test EVM.	A persistent network with trusted configuration and operational ownership.
Hash checks, ordered records, safe retries and tests.	Authentication, encrypted storage, key custody and operational monitoring.

Repository update: start with page 2 for the new six-field risk contract, API response wrapper and upload location. This edition is based on your uploaded SIH_26-main.zip, not an unseen live commit.

How to use this guide

Read	For
Pages 3-6	EVM, technology choices, the full flow and where data is stored.
Pages 7-11	Audit code, smart contract, connector, tamper checks and integration.
Pages 12-16	Colab cells, test evidence, future work, speaking notes and glossary.

Honest scope: we built an application on an EVM test environment, not a new blockchain protocol or a deployed multi-organization network. The design is useful as a prototype; production claims require additional evidence.

Repository changes you must know

What changed in v0.2.0, and what you should upload

Repository contract	Updated Member 5 behavior
riskScore.ts	Six required contributions: ocr, validation, tampering and face 0-20 each; barcode and watermark 0-10 each.
blockchainAudit.ts	AuditLogInput and confirmed AuditLogOutput field shapes are unchanged. The Solidity commitment contract does not need changing.
common.ts	Successful responses use success, data and requestId. api_adapter.py wraps pending ACKs and confirmed receipts; integration_types.ts imports root types.

Optional inputs do not mean optional output fields. Barcode and watermark inputs to the risk engine may be absent, but both numeric contributions are required in its output. Member 4 must define the missing-signal policy. We reject missing fields instead of inventing zero risk.

```
# Synthetic six-field risk result; contributions total 78.
{
  "score": 78, "decision": "flag",
  "summary": "Synthetic example: manual review recommended.",
  "breakdown": {
    "ocr": 18, "validation": 16, "tampering": 18,
    "face": 16, "barcode": 6, "watermark": 4
  }
}
```

The repository example still reports 78 while its contributions total 64. The adapter preserves that input and returns a mismatch warning. Existing four-field historical envelopes are not rewritten; the new schema is enforced for new submissions.

API integration: record_risk_response() accepts ApiResponse<RiskScoreOutput>, rejects upstream failures and passes data into the audit adapter. requestId is a correlation ID, not the stable event ID. success = true with data.logged = false means a successful local write, not blockchain confirmation.

Upload your whole Member 5 folder

Merge the supplied ZIP at the repository root. All additions are under modules/blockchain/. The final notebook belongs in its notebooks/ folder, and this PDF in docs/. Backend imports require the Python files too. Root contracts and teammate modules are not replaced.

Current backend blocker: modules/backend/document_verification.py fails parsing at line 9 (2.REFERENCE_FILE), with more numbered code fragments later. It does not implement the six-field risk pipeline. Member 4 must finish it before any full-system test can pass.

What is the EVM?

Start with the execution engine, then distinguish it from the network

EVM stands for Ethereum Virtual Machine. It is the software execution environment that runs compiled smart-contract instructions. Given the same starting state and transaction, compatible execution follows the same rules and produces the same result. [1]

A useful analogy

Solidity is the language used to write the rulebook. Compiled bytecode is the machine-readable rulebook. The EVM is the engine that executes it. Our AuditAnchor contract is one small rulebook about who may record a hash and in what order.

Term	Meaning in our project
Smart contract	AuditAnchor.sol: rules and stored values for audit commitments.
EVM	Executes those compiled rules; accepts valid state changes or reverts invalid ones.
Blockchain ledger	The network history and state against which we later check commitments.
Consensus	How a real network agrees on transaction order/history. It is not supplied merely by importing an EVM library.
Gas	A measure of execution work. A network can charge transaction fees using it; our demo uses test balances, not real money.

What does our EVM actually do?

When the worker calls **anchor(sequence, previous, digest)**, the EVM runs the contract checks. If the caller is authorized and the order is valid, it updates count, head and the stored hash. Otherwise the call reverts. The EVM never reads the passport image, performs OCR or recalculates the risk score.

What did we use to run it?

The notebook uses **EthereumTesterProvider** through Web3.py, with **eth-tester** and **Py-EVM** underneath. This gives us local accounts, test balances and executable contract transactions inside Python. Web3.py is the bridge used by our code; Py-EVM is the execution implementation. [2, 9]

Important distinction: this local test chain is disposable. Its real contract execution and transaction hashes do not imply that the records were published to Ethereum mainnet or agreed by independent organizations.

What we used, and why

A practical prototype choice, not a claim that blockchain is always necessary

Component	Role and reason for using it
Python 3.12	Matches the existing Python/Streamlit reference and supports a clean importable module.
SQLite + sqlite3	A small persistent local journal with transactions; no separate database server needed for the prototype.
hashlib / SHA-256	Fingerprints original file bytes and the complete event envelope. [5]
Solidity 0.8.24	Compiles the small audit contract into EVM bytecode.
Web3.py 7.16.0	Builds/signs transactions, reads the contract and retrieves receipts.
eth-tester + Py-EVM	Runs the local EVM demonstration without an external wallet/node.
Google Colab	Packages source, examples and tests into an accessible learning notebook. Runtime state is temporary. [8]

Does the blockchain choice survive a critical review?

For this SIH prototype: yes. It demonstrates executable rules, controlled writes and comparison against separately recorded commitments. It also keeps your component independent from changing AI/backend code. The EVM path gives us a small contract and a reusable Python connector.

For a single operator controlling everything: the case is weaker. If that operator can replace the database, reset the chain and change verifier configuration, the comparison is not independent. A signed append-only log with independently retained checkpoints may be simpler. That is an architectural alternative, not something this package implements.

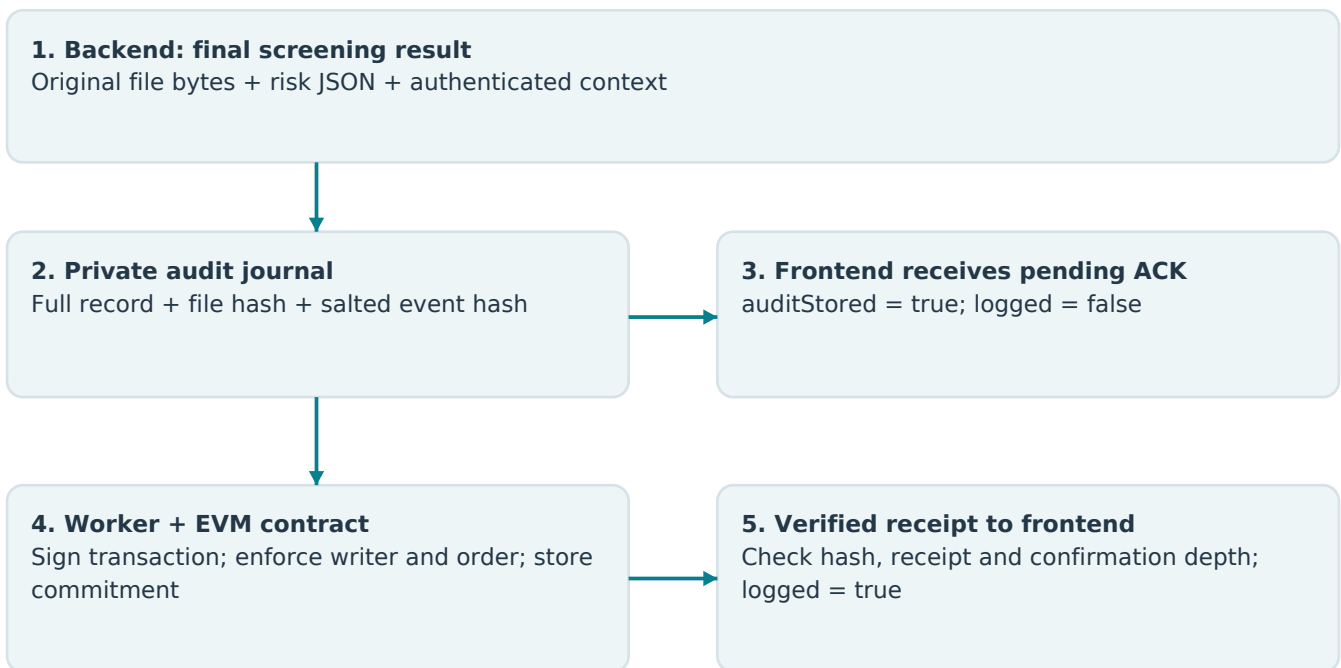
For multiple organizations: evaluate a permissioned EVM network or Fabric against actual governance needs. Fabric has identity-aware policies and endorsement mechanisms, but moving to it requires new chaincode/connector work. The word Fabric in the supplied contract was an example ledger label. [7]

Maintenance finding for future work

Py-EVM documentation identifies it as archived/read-only. Keep the tested dependency set for the demonstration, but evaluate a maintained node/runtime before extending it into a deployment. Passing the local tests does not establish future protocol support. [9]

One screening, from start to finish

The backend connects the AI result to your independent audit module



Follow the record

1. The backend retains the original upload bytes before image decoding or resizing. Members 1-3 produce their outputs; Member 4 builds and persists the final risk decision.
2. Your adapter validates the contracted JSON, hashes the original file and appends an envelope to SQLite. It records a server timestamp, context, a random salt and the previous event hash.
3. The adapter returns a local acknowledgement immediately. The application can show the risk result while clearly labeling blockchain anchoring as pending.
4. The worker finds the next unanchored record, signs a contract call and submits it. The smart contract only receives sequence, previous hash and current digest as inputs.
5. A read checks the commitment and mined receipt at the configured confirmation depth. Only then does the frontend receive the original AuditLogOutput with logged = true.

Why the worker is separate

Network delays should not block model inference. The journal retains work to retry; the UI distinguishes decision availability, audit storage and confirmed anchoring. A timeout is uncertainty, never permission to fabricate a receipt.

What data and hashes mean

Three different hashes and a deliberate on-chain/off-chain boundary

Value	What is hashed / what it proves
Document SHA-256	Original uploaded file bytes. Recomputing it checks whether the bytes match what was recorded.
Event SHA-256	Exact stored envelope JSON bytes, including document hash, risk input, IDs, timestamps, salt and previous hash. It binds the audit record together.
Transaction hash	Returned by the EVM chain for its transaction. It identifies the blockchain transaction; our Python SHA-256 helper does not manufacture it.

```
def sha256(data):  
    return hashlib.sha256(data).hexdigest()  
  
# Hash original file bytes, before image processing.  
document_sha256 = sha256(original_document_bytes)
```

SHA-256 produces a 256-bit digest, represented here as 64 hexadecimal characters. It is a fingerprint, not encryption and not proof that a document is genuine. Two visually identical files can have different bytes and therefore different digests. [5]

Data	Where our implementation keeps it
Passport image, live face, embeddings	Not stored by this audit package. Secure artifact storage belongs to the backend.
Risk score, decision, summary, checkpoint and actor	Inside the private off-chain journal; never separately published in our contract.
Random salt and original-file digest	Inside the off-chain envelope; included indirectly in the event commitment.
Sequence and event commitment	Stored in contract state; writer address and transaction metadata are also visible on-chain.

Salt: each new envelope gets 32 random bytes. This makes guessing a low-entropy record from its commitment harder while the envelope remains private. It does not encrypt SQLite or stop a reader who already has the record.

Privacy boundary: SQLite is plaintext. The backend must authorize access, protect backups and encrypt storage. Free-text summaries can contain identifying information; keeping them off-chain does not make them harmless.

Inside audit_trail.py

Validate, serialize, link and commit one durable audit event

What happens when append() is called?

Step	Code behavior and reason
1. Validate	Check IDs and lowercase SHA-256 format. Require a JSON object; reject an oversized event above 256 KiB.
2. Serialize the request	canonical() sorts JSON keys, removes extra separators and rejects NaN/Infinity. We preserve exact UTF-8 envelope bytes for later hashing.
3. Begin a transaction	BEGIN IMMEDIATE obtains the SQLite write transaction before selecting the next sequence. Concurrent writers cannot allocate the same next position. [6]
4. Check event_id	Same ID + identical request returns the existing record. Same ID + different request raises an error. This makes retries idempotent.
5. Build the envelope	Use the previous event hash and next sequence, plus schema, server recorded_at, random salt and request.
6. Hash and commit	Compute SHA-256 of the envelope string, insert the row and commit. Exceptions roll back; no successful storage ACK is returned.

```
envelope = canonical(dict(
    schema='member5.audit.v1', sequence=seq,
    previous_hash=previous,
    recorded_at=datetime.now(timezone.utc).isoformat(),
    salt=secrets.token_hex(32), request=json.loads(request))
event_hash = sha256(envelope.encode())
```

Why two IDs? verification_id groups a screening attempt; event_id identifies one event within it. One attempt can have upload, OCR, face and final-decision events. A repeat screening of the same document gets a new verification ID. Retry an existing event with the same ID and data.

What is stored? Each row has sequence, event_id, request_json, envelope_json and event_hash. The request is duplicated inside the envelope, and verification checks that both representations agree. history() filters events by screening attempt.

Guardrails, not magic

SQLite triggers reject ordinary UPDATE and DELETE statements, but a database administrator can drop them. verify_local() detects inconsistencies; independent blockchain comparison is what detects an anchored history that has been fully rewritten and rehashed.

The serializer is our versioned Python format, not RFC 8785. Other languages should hash the stored envelope string directly, rather than parse and reserialize it.

Inside AuditAnchor.sol

The smart contract has a narrow job: preserve an ordered list of commitments

State / function	Meaning
owner; writers[address]	The deployer starts as owner and authorized writer. Only the owner can add/revoke writers using setWriter().
hashes[sequence]	Maps each anchored sequence number to its bytes32 event digest.
count; head	Number of anchored entries and latest committed event digest.
Anchored event	An EVM log containing sequence, digest and writer; used to locate the original transaction receipt.

A **mapping** is a key-value lookup. **msg.sender** is the caller address. **require()** enforces a condition and reverts invalid changes. These are Solidity constructs used by our contract. [3]

```
function anchor(uint256 sequence, bytes32 previous,
               bytes32 digest) external {
    require(writers[msg.sender], "writer only");
    require(digest != bytes32(0), "zero digest");
    if (sequence > 0 && sequence <= count) {
        require(hashes[sequence] == digest,
               "conflicting retry");
        return;
    }
    require(sequence == count + 1 && previous == head,
           "out of order");
    hashes[sequence] = digest;
    head = digest;
    count = sequence;
    emit Anchored(sequence, digest, msg.sender);
}
```

Read it in order: reject unauthorized writers and the zero digest; accept an exact retry of an existing digest without a second event; otherwise require the next sequence and correct previous head. Store the new digest and emit its log. The excerpt preserves the supplied function logic, with line wrapping adjusted.

For an exact retry, the contract compares the stored digest and returns; it does not recheck the supplied previous hash. For a new record, it enforces previous == head. There is no method to replace an existing commitment.

What the contract cannot establish

It receives only opaque hashes. It cannot inspect the hidden envelope, verify the AI score or identify a human officer. It trusts authorized writers to construct honest future records. The owner address is immutable; there is no built-in ownership recovery or upgrade mechanism.

Inside blockchain.py and worker.py

Submission, receipt checks and safe retry behavior

EVMAnchor checks the expected chain ID and that code exists at the configured contract address. It uses the contract ABI, the description of function names and argument types, to encode calls. Code presence alone does not prove the address contains the reviewed contract; deployment configuration must be trusted.

The transaction path

```
tx = contract_function.build_transaction(transaction_fields)
signed = account.sign_transaction(tx)
tx_hash = w3.eth.send_raw_transaction(signed.raw_transaction)
receipt = w3.eth.wait_for_transaction_receipt(tx_hash, timeout=60)
```

This is an abbreviated view of submit(). The **private key** signs the request; the chain verifies the sender. The transaction **nonce** is the account sequence number. chainId binds the transaction to its network. Build/sign/send/receipt operations use Web3.py. [4]

Observed condition	Our behavior
Already on-chain, same digest	Wait for required confirmations instead of sending another transaction.
Signer has a pending transaction	Return pending_transaction; keep the same dedicated account under one worker.
Receipt status is 0	Raise a reverted-transaction error.
Receipt wait times out	Propagate uncertainty. The next run reads chain state before deciding what remains.
Receipt status is 1	Return mined metadata; this alone is not the final confirmed output contract.

snapshot() reads a block at the configured confirmation depth. **receipt_output()** checks the committed hash, original Anchored log, successful receipt and matching block hashes, then returns a mined timestamp and ledger identity. One confirmation means inclusion, not irreversible finality. **anchor_next()** verifies the journal against the confirmed chain count and submits the next pending row. The journal itself is the durable queue: rows beyond that count remain work to do. The worker handles at most one such row per invocation.

worker.py reads configuration from environment variables and uses a separate SQLite lock file to prevent overlapping workers on the same path. The lock is released after a crash. It is not a distributed lock or a shared-wallet nonce manager. The journal write lock is not held while waiting on the network.

How tampering is detected

Local consistency and independent anchoring answer different questions

Check	What the code examines
verify_local()	Contiguous sequences, previous hashes, envelope digest, matching event IDs and matching request copies.
verify_anchoring()	Local validation plus contract count, every anchored per-sequence digest and the contract head.
verify_document()	New file bytes versus the recorded document digest; optionally checks that the corresponding event is anchored.

Attack 1: someone changes a stored decision

An attacker changes flag to accept without updating the envelope digest. `verify_local()` recomputes the digest and rejects the mismatch. A simple hash chain catches this inconsistent edit.

Attack 2: someone rewrites the whole local hash chain

A database administrator drops the update trigger, changes the decision and recalculates each affected hash and link. The local chain can now pass its own consistency check. However, its digests differ from the already recorded contract hashes. `verify_anchoring()` raises a blockchain mismatch.

Attack 3: someone removes the end of the journal

If the removed events were anchored, the contract count is larger than the surviving local record count. Verification detects missing anchored history. If the removed tail had never been anchored, the ledger cannot prove that those records once existed.

What the Colab demonstration does

Cell 22 backs up the journal to a temporary copy, alters the final decision and recalculates the local hash chain. It shows that local consistency passes but blockchain comparison fails. The original journal is kept intact. This is stronger than only demonstrating that an unchanged stored hash no longer matches an edited field.

The remaining trust boundary

The verifier must trust the selected network, contract and RPC source. If the same attacker can replace both the journal and the ledger/configuration, a comparison against that replacement provides no independent assurance. A compromised writer can also submit false new records. Neither case is solved by a hash function.

Correction workflow: append a new `OFFICER_OVERRIDE` event with a reference and reason. Do not edit the old decision. The backend must authorize that override and provide the real actor identity.

How your team integrates it

Member 4 calls your adapter; Member 6 renders its status

```
from member5 import AuditTrail, record_verification

audit = AuditTrail(config.audit_database_path)
ack = record_verification(
    audit,
    {
        "documentId": internal_document_id,
        "checkpointId": authorized_checkpoint_id,
        "timestamp": persisted_decision_timestamp,
        "riskResult": final_risk_result,
    },
    event_id=persisted_event_id,
    verification_id=persisted_attempt_id,
    actor_id=authenticated_actor_id,
    original_document_bytes=original_bytes,
)
```

The names above are backend-supplied variables. In the old Streamlit reference, retain original bytes using `document_file.getvalue()` before decoding with OpenCV. Member 4 must create the new contracted risk output; old DOB/face status branches do not define a valid score by themselves.

Owner	Responsibility
Members 1-3	Produce OCR, tampering and face results; send them to Member 4.
Member 4	Validate and score; authenticate context; persist result and a retryable audit job together.
Member 5: you	Maintain audit module, validation adapter, contract, connector, worker and integrity checks.
Member 6	Connect the API/UI; show pending, unavailable or confirmed audit state correctly.

Input validation: score must be 0-100, OCR/validation/tampering/face contributions 0-20 and barcode/watermark 0-10, decision one of accept/flag/reject, and timestamp timezone-aware. Exact field names are enforced. A breakdown-total mismatch produces a warning while preserving the supplied result.

Local ACK: `auditStored = true` and `logged = false`. This means the adapter committed locally, not that it checked current chain status. Even a duplicate local ACK is not a chain-status query.

Confirmed output: `get_audit_output()` returns `txHash`, `blockNumber`, `timestamp`, `logged` and `ledger`; `None` means pending. Exceptions must not be shown as success. The output timestamp comes from the mined block, not the input screening timestamp.

The missing atomic boundary is backend work

Member 5 cannot atomically update an unknown backend database and SQLite together. Member 4 should persist the risk result plus a durable integration job in one backend transaction, then retry that job with the same event ID and data until acknowledged.

Your Colab notebook, cell by cell

Use this page while demonstrating the prototype

Cells	What happens / what to explain
1-3	Create folders; restore embedded support files; install dependencies. No earlier ZIP is required.
4-5	Write <code>api_adapter.py</code> for the shared response envelope, then <code>audit_trail.py</code> for the journal.
6-7	Write <code>contracts.py</code> and <code>integration.py</code> : six-field validation and event adapters.
8-11	Write blockchain connector, worker, CLI and package exports.
12-13	Write Solidity source; import modules; check the compiled artifact source hash.
14	Verify the frozen TypeScript contract snapshot; print all six contribution limits.
15-16	Define the callable; create local test accounts, contract and matching journal.
17	Define synthetic six-field <code>AuditLogInput</code> and original document bytes.
18	Record upload and final decision using the upstream <code>ApiResponse</code> wrapper. The ACK is pending.
19-20	Anchor records, verify the receipt and wrap <code>AuditLogOutput</code> for Member 6.
21-22	Inspect history/file hashes; rewrite a copy of the journal to demonstrate tamper detection.
23	Run all 39 tests; do not accept a skipped EVM suite as full verification.
24-25	Optional: upload a sample document plus complete <code>AuditLogInput</code> JSON and record it.
26	Optional: read-only connection to an existing persistent EVM deployment.
27	Export source with <code>modules/blockchain/</code> paths. The full delivered ZIP also includes notebook/PDF.

Safe demo order: run the default cells in order. Explain the pending ACK, then show the receipt, then show the tamper test. Optional switches are false by default, so Run all needs no uploaded document or external wallet.

Reruns: the same kernel reuses its demo chain/journal. A new kernel creates a new matched pair. Reusing an event ID with changed input correctly raises a conflict. For a new screening, create new IDs. Source-writing cells overwrite their named files; edit those cells and restart imports/runtime to apply changes.

Export is not evidence backup: the ZIP contains code, not the temporary chain or private journal. Colab runtime storage has a limited lifetime. Preserve an operational journal separately using a consistent SQLite backup and a persistent network. [8]

What has been proved so far?

Test evidence, security coverage and limits

Recorded verification result

All 39 package tests passed: 16 audit tests, 12 repository-contract tests and 11 local-EVM tests. The 27-cell notebook flow executed locally through in-process IPython in an isolated environment, including its dependency-installation cell. TypeScript fixtures also compiled against the uploaded root contracts. These are prototype verification results, not a security audit or production certification.

Evidence from tests	What it supports
Concurrent append and same-ID retries	Ordered local writes and duplicate-event handling.
Validation/ranges and conflicting payloads	Invalid input is rejected; existing records are not silently overwritten.
Edited, rehashed and truncated journals	Local corruption and disagreement with retained commitments are detected in the tested cases.
Real local-EVM execution	Compiled contract rejects unauthorized/out-of-order/conflicting writes and supports exact retries.
Receipt and confirmation tests	Final output fields use actual chain data; extra confirmation depth prevents premature reporting.
Timeout after broadcast and network failure	Pending work survives; chain reconciliation avoids a duplicate submission in the tested timeout scenario.

Security responsibilities that are still outside this module

The backend must establish human identity, authorize checkpoint/case access and officer overrides, secure HTTP communication and protect documents. The signer key must remain server-side. The contract authorizes a wallet; it does not authenticate an individual officer. A database writer without chain access cannot silently replace a retained anchored commitment through this contract. A compromised authorized signer can still append false future commitments until revoked. Storage encryption, backup access and network governance remain operational requirements.

Do not overstate the tests

We did not test the future rewritten backend, a hosted Colab session with its interactive dialogs, a persistent HTTP-RPC deployment, independent-node consensus, checkpoint load or key-custody procedures. Thirty-nine passing tests are not 100% security or AI accuracy.

Notable implementation limits: verification loads/scans the journal and queries anchored entries; one contract serializes one journal; the worker lock is local; stuck-transaction gas replacement is manual; the owner is immutable; confirmation checks do not eliminate all reorganization or malicious-RPC risk.

What can we improve in the future?

Prioritize evidence and reliable integration before adding features

Priority	Change	How to know it is better
1. Team integration	Connect the new risk output; add durable backend jobs, authenticated actor context and all actual stage events.	A screening can be traced from upload to decision; killing either process does not silently lose the integration job.
2. Persistent test deployment	Move beyond the disposable Py-EVM session. Evaluate a maintained node/runtime; pin and re-test the selected dependency/network combination. [9]	Restart services and still verify the same historical records against the same deployed contract.
3. Independent governance	Define who runs nodes, who can add writers, how keys are rotated and how an investigator pins the ledger identity.	A compromised single database/operator cannot substitute all trusted evidence.
4. Operational security	Separate owner/service keys; add secret management, access controls, encrypted storage/backups and monitoring.	Revoke a compromised writer, restore a consistent backup and generate a clear incident alert.
5. Throughput	Index receipts/history, use a shared durable queue for multiple hosts, and measure latency before optimizing.	Measure append time, confirmation delay, failure recovery and memory at representative load.
6. Batched commitments	Consider Merkle-root batches, versioned canonicalization and independent verification tooling.	An investigator can verify one event using its proof and the anchored batch root.

What is a Merkle-root batch?

Instead of one transaction per event, combine many event hashes into a tree of hashes and anchor its final root. Keep a proof showing how each event contributes to that root. This could reduce transaction volume, but it adds proof storage, batching delay and failure-handling complexity. It is a proposed design, not implemented code.

Would Fabric be better?

Possibly, if the real requirement is named organizations jointly endorsing updates with identity-based policies. Fabric supports those mechanisms. [7] An EVM network may be preferable if preserving our Solidity/Web3 integration matters more. Decide from governance, privacy and operations requirements; neither option automatically makes the AI correct.

First improvement to make: complete the backend integration and durable recovery test. A larger blockchain framework cannot fix an audit event that the backend never sends.

Explain it to your team or judges

A short speaking script, followed by the questions worth preparing for

A 60-second explanation you can use

My Member 5 component records the history of document screening. The AI and risk engine decide the result; my code preserves the record of that result. First, we hash the original file and place its fingerprint, result and context in a private SQLite audit journal. We hash the complete event and send that commitment to a Solidity smart contract. The EVM executes the contract rules: only authorized writers can append entries in order. Later, we compare the stored event with the blockchain commitment. If someone changes and rehashes the local record, the old commitment still reveals the mismatch. Our Colab version demonstrates this on a local EVM. A real deployment still needs a persistent network, secure keys and backend integration.

Questions and direct answers

Did you build a blockchain from scratch?

No. We built the audit application and smart contract, using an existing EVM implementation to execute it.

Why not store everything on-chain?

Our full record contains private screening details. We keep it off-chain and anchor only a commitment; this also keeps the contract small.

Can blockchain detect a fake passport?

Not in this design. AI/validation modules assess forgery. Our checks detect changes to recorded bytes or history, not truth at the time of entry.

Why do we need both SQLite and blockchain?

SQLite retains readable audit detail. The ledger retains a separate commitment used for comparison. The security value depends on the ledger being independently trustworthy.

What if the internet or node fails?

The local event stays pending. A worker retries and reconciles with chain state. We never label an unverified entry as confirmed.

Can an officer change a decision?

The backend can authorize a new override event with a reason and a reference to the old event. It should not edit the previous record.

Is this tamper-proof?

Use tamper-evident for anchored history under stated trust assumptions. A reset local chain, compromised signer or omitted event is outside that absolute claim.

What exactly is your personal contribution?

The audit journal, hashing, input adapter, Solidity anchor rules, blockchain connector, recovery behavior, verification checks and demonstrable tests.

Glossary and source notes

Keep this page open when explaining unfamiliar terms

Term	Plain meaning
EVM / ABI	Ethereum Virtual Machine: executes contract code. Application Binary Interface: describes how to call contract functions.
SHA-256 / digest	Secure Hash Algorithm with a 256-bit result; digest is its fingerprint output.
Audit event / EVM event	An audit event is our full record of an action. An EVM event is a contract-emitted log associated with a transaction.
Transaction / block	A signed chain request / a grouping of transactions in chain history. One audit event is not inherently one blockchain block.
Anchor / commitment	The event digest recorded on-chain for later comparison with its private source record.
Nonce / idempotency	Signer transaction sequence / the property that retrying the same event does not create a new logical record.
RPC / receipt	Remote Procedure Call interface to a node / the chain response describing transaction execution and inclusion.
On-chain / off-chain	Held in the blockchain environment / held outside it, such as the private journal.
Salt / confirmation	Random envelope input / evidence of block inclusion and depth, subject to that network's rules.
PII / WAL	Personally Identifiable Information / Write-Ahead Logging, the SQLite journal mode used here.

Sources and provenance

Implementation descriptions come from the delivered modules/blockchain package (v0.2.0), AuditAnchor.sol, BACKEND_HANDOFF.md, TEST_REPORT.md and SIH_Member5_Blockchain_Audit_Colab.ipynb. This guide describes the repository-reviewed version. Root TypeScript files from the uploaded SIH_26-main.zip are preserved in contract_reference/ with hashes. The archive had no Git commit ID; no live GitHub revision is claimed.

[1] [Ethereum.org: EVM](https://ethereum.org/en/developers/docs/contracts/abi/)

[2] [Web3.py: providers and EthereumTesterProvider](#)

[3] [Solidity: introduction to smart contracts](#)

[4] [Web3.py: transaction and receipt API](#)

[5] [Python: hashlib](#)

[6] [SQLite: transactions](#)

[7] [Hyperledger Fabric: policies](#)

[8] [Google Colab: FAQ and runtime limitations](#)

[9] [Py-EVM: introduction and archive notice](#)

Official references checked on 13 September 2026. Source/API explanations are paraphrased. Technology-choice judgments and the future roadmap are recommendations for this prototype, not measured deployment results.